```
Array
(
    [0] => Iain M. Banks
    [1] => I am not an animal brain, I am not even some attempt to produce
    an AI through software running on a computer
)
```

The code above illustrates an obvious problem with the CSV format: what happens when the delimiter character is also field data? Both quotes are full sentences with grammatically correct commas, but as we can see from parsing `$csvRow1`, we end up with three strings in the output array. `$csvRow2` solves the problem using the field enclosure method described in RFC 4180 (and was in conventional use prior to the RFC's publication); here we enclose the field data with double-quote meta-characters. We can see from the output that this gives correct results: the second field is returned as a single string, and the comma survives intact. Oh? What's that I hear you say? Yes, absolutely correct. The next problem we now have is handling the enclosure character itself. Well spotted! The RFC and most tools that create CSV data (MS Excel, Google Docs…) make use of a second instance of the enclosure character as an escape sequence, as we can see here:

```
$csvRow3 = 'Iain M. Banks,';
$csvRow3 .= '"""Jernau Gurgeh,"" the machine said, making a sighing noise,
""a guilty system recognizes no innocents..."""';

print_r(str_getcsv($csvRow3, escape: '\\'));

Output:
Array
(
    [0] => Iain M. Banks
    [1] => "Jernau Gurgeh," the machine said, making a sighing noise, "a
           guilty system recognizes no innocents..."
)
```

It's ugly syntax, but it works. This is how the popular tools implement it, and it's how the RFC specifies it should work. Interestingly, the official documentation for `str_getcsv()` states that a backslash character can be used to escape the enclosure character, which does work to some extent. Let's throw some awful data at this function and see what happens.